

**SRS**

# SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

**Dự án:** Hệ thống Quản lý Trạm đổi pin xe điện (Volt Swap Project)

**Phiên bản:** 1.0 (Bản tinh gọn)

## 1. GIỚI THIỆU (INTRODUCTION)

### 1.1. Mục đích (Purpose)

Tài liệu này xác định các yêu cầu phần mềm cho dự án Hệ thống Quản lý Trạm đổi pin xe điện (Volt Swap). Hệ thống số hóa và tự động hóa quy trình quản lý gói cước, đặt lịch hẹn, vận hành trạm đổi và quản lý vòng đời của pin xe điện, mang lại trải nghiệm đổi pin nhanh chóng cho người dùng.

### 1.2. Phạm vi (Scope)

Phần mềm tập trung giải quyết bài toán quản lý trạng thái thiết bị (trạm, trụ, khe cắm), theo dõi dung lượng và sức khỏe pin, quy trình đặt lịch đổi pin (Booking) và tích hợp cổng thanh toán trực tuyến. Hệ thống áp dụng cho cả mô hình tự phục vụ và đổi pin có nhân viên hỗ trợ.

### 1.3. Định nghĩa & Viết tắt

- EV (Electric Vehicle):** Xe điện.
- SoH (State of Health):** Trạng thái sức khỏe pin (tỷ lệ phần trăm dung lượng còn lại so với ban đầu).
- Pillar / Slot:** Trụ sạc và khe cắm pin tại các trạm đổi.
- BaaS (Battery-as-a-Service):** Mô hình cung cấp pin dưới dạng dịch vụ thuê bao.

## 2. MÔ TẢ TỔNG QUAN (OVERALL DESCRIPTION)

### 2.1. Phân quyền người dùng (User Roles)

Để hệ thống vận hành trơn tru nhưng không quá phức tạp, phần mềm được thiết kế với 3 vai trò (Roles) cốt lõi:

**ADMIN (Quản trị hệ thống & Điều phối)**

Quản trị viên cấp cao. Quản lý toàn bộ danh mục trạm, trụ, gói cước, và tài khoản hệ thống. Theo dõi tổng quan vận hành, điều phối pin liên trạm và quản lý báo cáo doanh thu, giao dịch.

#### **STAFF (Nhân viên vận hành trạm)**

Quản lý trực tiếp pin trong kho và các trụ sạc tại trạm được gán.

Xử lý lịch hẹn (Booking), hỗ trợ đổi pin thủ công cho tài xế và gửi yêu cầu hỗ trợ lên Admin.

#### **DRIVER (Tài xế / Khách hàng)**

Sử dụng ứng dụng di động để đăng ký gói cước, quản lý xe cá nhân.

Tìm kiếm trạm trên bản đồ, đặt lịch hẹn trước, thanh toán và theo dõi lịch sử đổi pin.

## **2.2. Môi trường hoạt động**

- **Frontend:** Ứng dụng di động (cho Tài xế) và giao diện Web-based (cho Admin và Staff).
- **Backend:** RESTful API tích hợp cổng thanh toán trực tuyến (VNPay).
- **Database:** Cơ sở dữ liệu quan hệ (Relational Database) với 19 bảng quản lý thực thể.

# **3. YÊU CẦU CHỨC NĂNG (FUNCTIONAL REQUIREMENTS)**

## **3.1. Quản trị Hệ thống & Người dùng (Auth & Master Data)**

**FR\_1.1. Xác thực & Phân quyền:** Người dùng đăng nhập bằng Email/Số điện thoại và Mật khẩu. Giao diện hiển thị các chức năng tương ứng theo phân quyền hệ thống.

**FR\_1.2. Quản lý Tài khoản & Trạm:** ADMIN có thể tạo mới, cập nhật, hoặc vô hiệu hóa tài khoản nhân viên, khách hàng và gán nhân viên vào quản lý trạm cụ thể.

**FR\_1.3. Cấu hình Gói cước:** ADMIN thiết lập và cập nhật các gói dịch vụ thuê pin (Subscription Plans) bao gồm thời hạn, định mức và mức phí cọc.

## **3.2. Đăng ký & Đặt lịch (Subscriptions & Booking)**

**FR\_2.1. Đăng ký gói cước & Đặt cọc:** DRIVER đăng ký gói dịch vụ và thực hiện đặt cọc 400.000 VNĐ qua cổng thanh toán VNPAY để kích hoạt quyền thuê pin.

**FR\_2.2. Đặt lịch hẹn (Booking):** DRIVER chọn trạm trên bản đồ và tạo lịch hẹn đổi pin trước tối thiểu 1 giờ. Trạng thái lịch hẹn khởi tạo là PENDING.

**FR\_2.3. Kiểm tra điều kiện:** Hệ thống chỉ cho phép đặt lịch mới khi DRIVER không có lịch hẹn hoặc lượt đổi pin nào đang bỏ dở.

## **3.3. Vận hành Trạm & Đổi pin (Station Ops & Swap)**

**FR\_3.1. Tiếp nhận và Đổi pin:** Khi tài xế đến trạm, STAFF hoặc hệ thống tự động kiểm tra thông tin đặt lịch, tiến hành mở khe cắm để nhận pin cạn và trả ra pin có dung lượng đạt chuẩn 100%.

**FR\_3.2. Quản lý khe cắm (Slot):** Mỗi trụ sạc có 20 khe cắm, hệ thống bắt buộc luôn duy trì trống tối thiểu 4 khe để sẵn sàng tiếp nhận pin cạn trả về từ khách hàng.

**FR\_3.3. Thu phí đổi pin:** Ghi nhận phí dịch vụ đổi pin tự phục vụ cố định là 10.000 VNĐ/lần và trừ vào tài khoản ứng dụng hoặc thanh toán trực tuyến.

### 3.4. Quản lý Chất lượng & Bảo trì (Battery Quality & Maintenance)

**FR\_4.1. Tự động kiểm tra chất lượng:** Khi nhận pin cạn vào khe cắm, hệ thống tự động đo chỉ số Sức khỏe pin (SoH).

**FR\_4.2. Chuyển trạng thái bảo trì:** Nếu chỉ số SoH của viên pin giảm xuống dưới 65%, hệ thống tự động khóa khe cắm đó và chuyển pin sang trạng thái cần bảo trì, không cho phép phân phối lại cho khách hàng.

## 4. YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS)

### 4.1. Hiệu suất (Performance)

Hệ thống phải có khả năng xử lý thông tin trạng thái sạc theo thời gian thực đồng thời cho hàng trăm viên pin trên toàn mạng lưới trạm đổi mà không xảy ra tình trạng trễ đồng bộ dữ liệu lịch hẹn.

### 4.2. Tính toàn vẹn dữ liệu (Data Integrity)

Mọi giao dịch thanh toán tiền cọc, phí đổi pin và nhật ký di chuyển pin giữa các trạm (Transfer Logs) phải được ghi nhận đầy đủ, bất biến và không cho phép chỉnh sửa hoặc xóa thủ công để phục vụ công tác kiểm toán.

### 4.3. Giao diện (UI/UX)

Giao diện ứng dụng di động cho tài xế cần tích hợp bản đồ trực quan, hiển thị rõ số lượng pin khả dụng tại mỗi trạm bằng mã màu (Ví dụ: Trạm còn nhiều pin - Xanh lá, Trạm sắp hết pin - Vàng, Trạm không có pin/Bảo trì - Đỏ).

## 5. MÔ HÌNH TRẠNG THÁI (STATE MACHINES)

### 5.1. Vòng đời Lịch hẹn (Appointment / Booking)

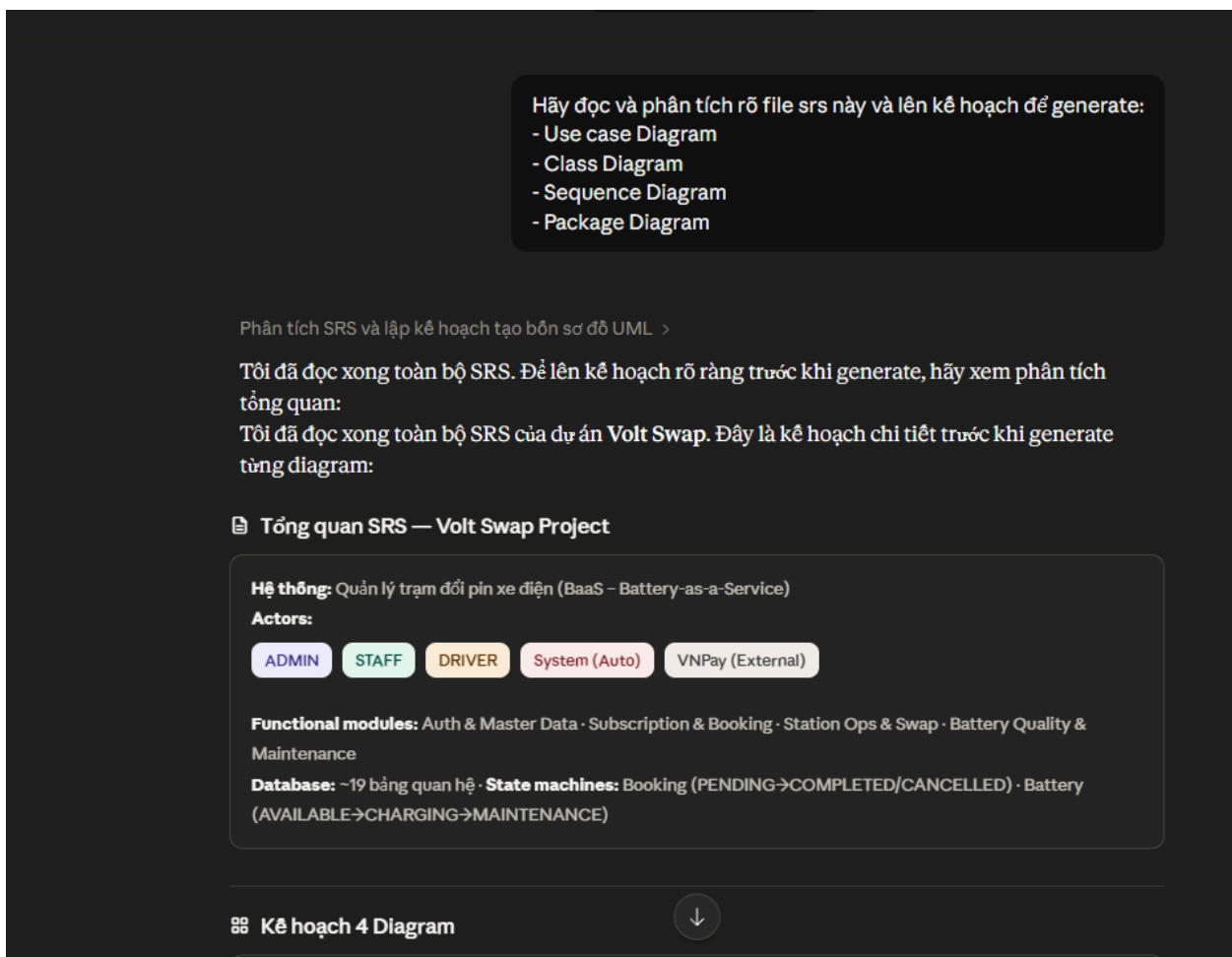
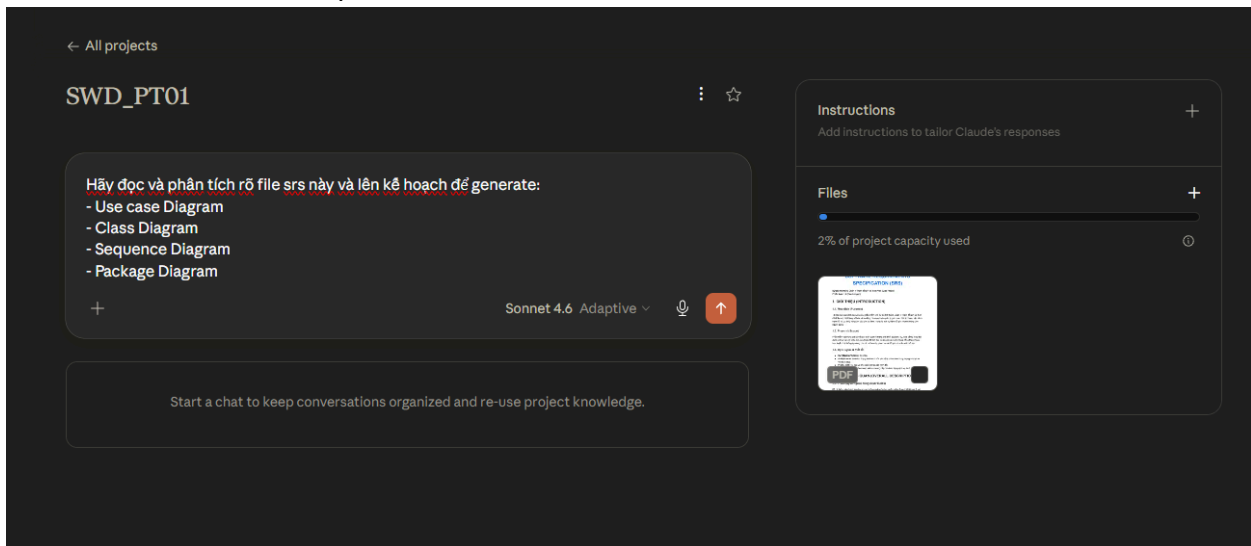
PENDING (Chờ đổi pin) → COMPLETED (Đã đổi pin thành công) hoặc CANCELLED (Hủy bỏ tự động nếu quá 1 giờ tài xế không tới trạm).

### 5.2. Vòng đời Trạng thái Pin (Battery State)

AVAILABLE (Sẵn sàng đổi) → CHARGING (Đang sạc trong khe) → MAINTENANCE (Bảo trì - tự động kích hoạt khi SoH < 65%).

# Use case diagram

Gửi file SRS để AI có thể phân tích:



oke hãy bắt đầu với Use Case Diagram, tôi muốn vẽ bằng PlantUML để có thể giúp tôi viết các tương tác để tạo diagram

Rất tốt. Thay vì render diagram trực tiếp, tôi sẽ tạo cho bạn code PlantUML hoàn chỉnh kèm một widget để xem và copy, dựa hoàn toàn trên SRS đã đọc.

### Use Case Diagram — PlantUML code

Copy code

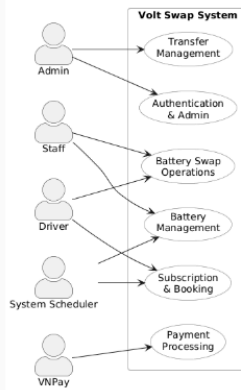
#### PLANTUML SOURCE

```
@startuml Volt_Swap_UseCase
    skinparam actorStyle awesome
    skinparam packageStyle rectangle
    skinparam usecase {
        BackgroundColor White
        BorderColor #888780
        ArrowColor #5F5E5A
    }
    skinparam actor {
        BackgroundColor White
        BorderColor #888780
    }
```

## 1. Overview Use Case Diagram

```
@startuml VoltSwap_Overview_UseCase
    left to right direction
    skinparam actorStyle awesome
    skinparam packageStyle rectangle
    skinparam shadowing false
    skinparam usecase {
        BackgroundColor #FFFFFF
        BorderColor #888080
    }
    skinparam package {
        BackgroundColor #F8F8F8
        BorderColor #888080
    }
    title Volt Swap - Overview Use Case Diagram
    actor Admin
    actor Staff
    actor Driver
    actor "VNPay" as VNPAY
    actor "System Scheduler" as SYSTEM
    rectangle "Volt Swap System" {
        usecase "Authentication/vn Admin" as UC_ADMIN
        usecase "Subscription/vn Booking" as UC_BOOK
        usecase "Battery Swap/vnOperations" as UC_SWAP
        usecase "Battery/vnManagement" as UC_BATTERY
        usecase "Transfer/vnManagement" as UC_TRANSFER
        usecase "Payment/vnProcessing" as UC_PAYMENT
    }
    Admin --> UC_ADMIN
    Admin --> UC_TRANSFER
    Staff --> UC_SWAP
    Staff --> UC_BATTERY
    Driver --> UC_BOOK
    Driver --> UC_SWAP
    VNPAY --> UC_PAYMENT
    SYSTEM --> UC_BOOK
    SYSTEM --> UC_BATTERY
enduml
```

#### Volt Swap - Overview Use Case Diagram



## 2. Authentication & Administration Use Case Diagram



```

partial VoltSwap_Admin_UseCase
left to right direction
skinparam actorStyle awesome
skinparam packageStyle rectangle
skinparam shadowing false

title Volt Swap - Authentication & Administration

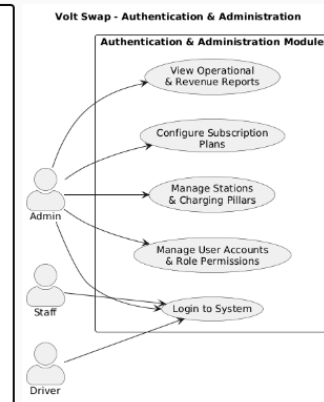
actor Admin
actor Staff
actor Driver

rectangle "Authentication & Administration Module" {
    usecase "Login to System" as UC_LOGIN
    usecase "Manage User Accounts\& Role Permissions" as UC_ACCOUNT
    usecase "Manage Stations\& Charging Pillars" as UC_STATION
    usecase "Configure Subscription\& Plans" as UC_PLAN
    usecase "View Operational\& Revenue Reports" as UC_REPORT
}

Admin --> UC_LOGIN
Admin --> UC_ACCOUNT
Admin --> UC_STATION
Admin --> UC_PLAN
Admin --> UC_REPORT

Staff --> UC_LOGIN
Driver --> UC_LOGIN
}enduml

```



### 3. Subscription & Booking Use Case Diagram

```
startuml
    VoltSwap_Subscription_Booking
    left to right direction
    skinparam actorStyle awesome
    skinparam packageStyle rectangle
    skinparam shadowing false

    title Volt Swap - Subscription & Booking

    actor Driver
    actor "VMPay Gateway" as VMPay
    actor "System Scheduler" as SYSTEM

    rectangle "Subscription & Booking Module" {
        usecase "Register Subscription\\nPlan" as UC_SUBSCRIBE
        usecase "Pay Deposit\\nvia VMPay" as UC_DEPOSIT
        usecase "Search Stations\\non Map" as UC_MAP
        usecase "Book Battery\\nSwap Appointment" as UC_BOOKING
        usecase "View Battery\\nSwap History" as UC_HISTORY
        usecase "Auto-Cancel\\nExpired Booking" as UC_CANCEL
    }

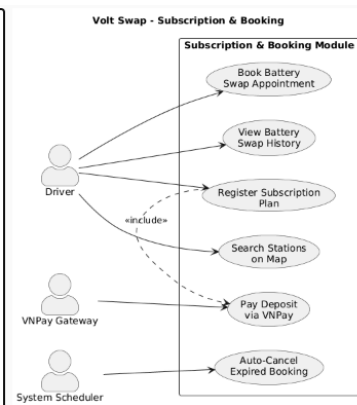
    Driver --> UC_SUBSCRIBE
    Driver --> UC_MAP
    Driver --> UC_BOOKING
    Driver --> UC_HISTORY

    VMPay --> UC_DEPOSIT

    SYSTEM --> UC_CANCEL

    UC_SUBSCRIBE -> UC_DEPOSIT : <<include>>

enduml
```



#### 4. Battery Swap Operations Use Case Diagram

```

partofall VoltSwap_Swap_Operations

left to right direction
skipparam actorStyle awesome
skipparam packageStyle rectangle
skipparam shadowing false

title Volt Swap - Battery Swap Operations

actor Driver
actor Staff
actor "VINWay Gateway" as VINWAY

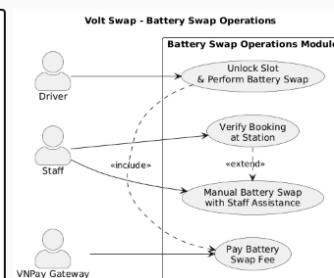
rectangle "Battery Swap Operations Module" {
    usecase "Verify Booking/Start Station" as UC_VERIFY
    usecase "Unlock Slot/Perform Battery Swap" as UC_SWAP
    usecase "Manual Battery Swap/with Staff Assistance" as UC_MANUAL
    usecase "Play Battery/InSwap Fee" as UC_PAYMENT
}

Driver --> UC_SWAP
Staff --> UC_VERIFY
Staff --> UC_MANUAL
VINWAY --> UC_PAYMENT

UC_SWAP --> UC_PAYMENT : <<include>>
UC_VERIFY --> UC_MANUAL : <<extend>>

@enduml

```

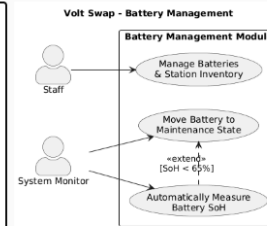


## 5. Battery Management Use Case Diagram

```

@startuml
    VoltSwap_Battery_Management
    left to right direction
    skinparam actorStyle awesome
    skinparam packageStyle rectangle
    skinparam shadowing false
    title Volt Swap - Battery Management
    actor Staff
    actor "System Monitor" as SYSTEM
    rectangle "Battery Management Module" {
        usecase "Manage Batteries\n& Station Inventory" as UC_BATTERY
        usecase "Automatically Measure\nBattery SoH" as UC_SOH
        usecase "Move Battery to\nMaintenance State" as UC_MAINT
    }
    Staff --> UC_BATTERY
    SYSTEM --> UC_SOH
    SYSTEM --> UC_MAINT
    UC_SOH --> UC_MAINT : <<extend>>[SoH < 65%]
    @enduml

```

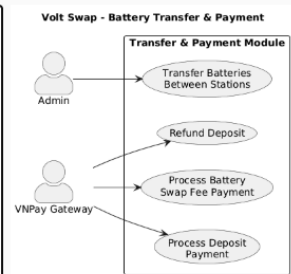


## 6. Battery Transfer & Payment Use Case Diagram

```

@startuml
    VoltSwap_Transfer_Payment
    left to right direction
    skinparam actorStyle awesome
    skinparam packageStyle rectangle
    skinparam shadowing false
    title Volt Swap - Battery Transfer & Payment
    actor Admin
    actor "VNPay Gateway" as VNPAY
    rectangle "Transfer & Payment Module" {
        usecase "Transfer Batteries\nbetween Stations" as UC_TRANSFER
        usecase "Process Deposit\nPayment" as UC_DEPOSIT
        usecase "Process Battery\nSwap Fee Payment" as UC_SWAPFEE
        usecase "Refund Deposit" as UC_REFUND
    }
    Admin --> UC_TRANSFER
    VNPAY --> UC_DEPOSIT
    VNPAY --> UC_SWAPFEE
    VNPAY --> UC_REFUND
    @enduml

```



# Class diagram

Giờ hãy tiếp tục với class diagram

## Class Diagram — PlantUML (Full)

Copy code

```
@startuml Volt_Swap_ClassDiagram

skinparam classAttributeIconSize 0
skinparam class {
    BackgroundColor #F5F5F0
    BorderColor #888780
    ArrowColor #5F5E5A
    FontSize 13
    HeaderBackgroundColor #E8E6DE
}

skinparam stereotypeCBackgroundColor #FAEEDA
skinparam stereotypeABackgroundColor #E1F5EE
```

## 1. User & Subscription Domain

```
@startuml VoltSwap_User_Subscription
skinparam classAttributeIconSize 0
skinparam linetype ortho

title Volt Swap - User & Subscription Domain

enum Role {
    ADMIN
    STAFF
    DRIVER
}

enum DepositStatus {
    PENDING
    PAID
    REFUNDED
}

class User {
    * id : UUID
    * email : String
    * phone : String
    - passwordHash : String

    * role : Role
    * isActive : Boolean
    * createdAt : DateTime
}

class SubscriptionPlan {
    * id : UUID
    * name : String
    * durationDays : Integer
    * swapQuota : Integer
    * depositAmount : Decimal
    * swapFee : Decimal
    * isActive : Boolean
}

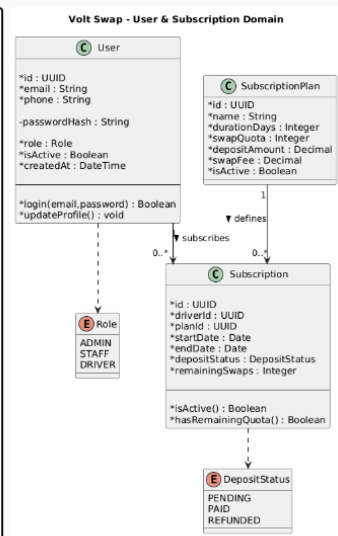
class Subscription {
    * id : UUID
    * driverId : UUID
    * planId : UUID
    * startDate : Date
    * endDate : Date
    * depositStatus : DepositStatus
    * remainingSwaps : Integer

    * isActive() : Boolean
    * hasRemainingQuota() : Boolean
}

User "1" --> "0..*" Subscription : subscribes
SubscriptionPlan "1" --> "0..*" Subscription : defines

User ..> Role : 
Role --> User : 
Subscription ..> DepositStatus : 
DepositStatus --> Subscription : 

enduml
```



## 2. Booking Domain



## 4. Payment Domain



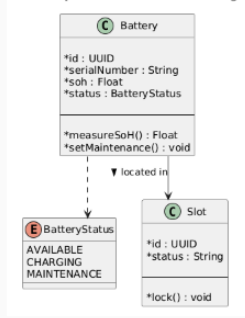
## 5. Battery Transfer Domain



## 6. Maintenance & Monitoring Domain

```
@startuml
    VoltSwap_Maintenance_Domain
    classDiagram
        class BatteryStatus {
            AVAILABLE
            CHARGING
            MAINTENANCE
        }
        class Battery {
            *id : UUID
            *serialNumber : String
            *soh : Float
            *status : BatteryStatus
        }
        class Slot {
            *id : UUID
            *status : String
        }
        Battery --> Slot : located in
        Battery ..> BatteryStatus
    
```

**Volt Swap - Maintenance & Monitoring**



# Sequence Diagram



oke hãy bắt đầu với Sequence Diagram, tôi muốn vẽ bằng PlantUML để có thể giúp tôi viết các tương tác để tạo diagram

Tốt. Ta sẽ bắt đầu bằng Sequence Diagram quan trọng nhất của hệ thống:

## FEFO Outbound & Delivery Flow

Đây là core business w  Ask ChatGPT ộ SRS vì nó chứa:

- Order lifecycle
- Inventory deduction
- FEFO allocation
- Batch management
- Immutable inventory ledger
- Delivery confirmation

## 1. Mục tiêu của Sequence Diagram này

Diagram sẽ mô tả flow:

APPROVED Order

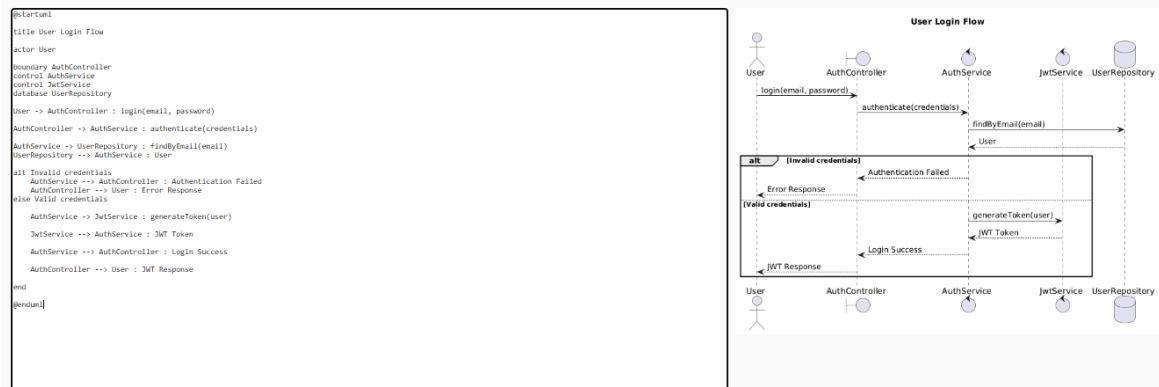
- > Export Inventory
- > FEFO Batch Allocation
- > Deduct Batch Quantity
- > Create Inventory Transactions
- > Delivery
- > Store Confirm Receive

## 2. Actors & Components

### External Actors

ADMIN\_KITCHEN  
STORE\_STAFF

### 1. SEQUENCE 1 — User Login Flow

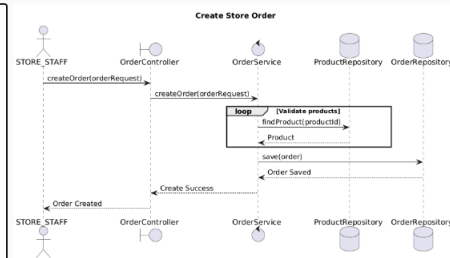


### 2. SEQUENCE 2 — Create Store Order

```

@startuml
title Create Store Order
actor STORE_STAFF
boundary OrderController
control OrderService
database ProductRepository
database OrderRepository
STORE_STAFF ->> OrderController : createOrder(orderRequest)
OrderController ->> OrderService : createOrder(orderRequest)
loop Validate products
    OrderService ->> ProductRepository : findProduct(productId)
    ProductRepository -->> OrderService : Product
end
OrderService ->> OrderRepository : save(order)
OrderRepository -->> OrderService : Order Saved
OrderService -->> OrderController : Create Success
OrderController -->> STORE_STAFF : Order Created
@enduml

```

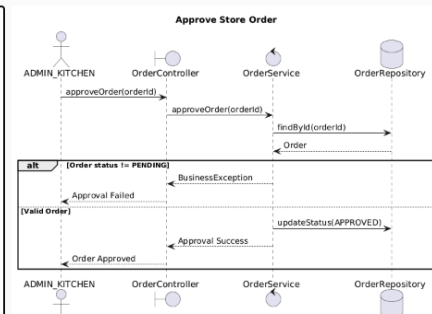


### 3. SEQUENCE 3 — Approve Store Order

```

@startuml
title Approve Store Order
actor ADMIN_KITCHEN
boundary OrderController
control OrderService
database OrderRepository
ADMIN_KITCHEN ->> OrderController : approveOrder(orderId)
OrderController ->> OrderService : approveOrder(orderId)
OrderService ->> OrderRepository : findById(orderId)
OrderRepository -->> OrderService : Order
alt Order status != PENDING
    OrderService -->> OrderController : BusinessException
    OrderController -->> ADMIN_KITCHEN : Approval Failed
else Valid Order
    OrderService ->> OrderRepository : updateStatus(APPROVED)
    OrderService -->> OrderController : Approval Success
    OrderController -->> ADMIN_KITCHEN : Order Approved
end
@enduml

```

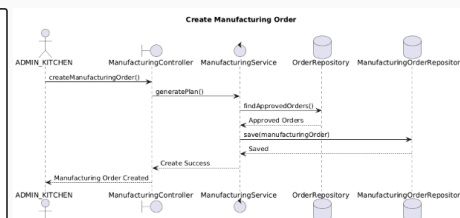


### 4. SEQUENCE 4 — Create Manufacturing Order

```

@startuml
title Create Manufacturing Order
actor ADMIN_KITCHEN
boundary ManufacturingController
control ManufacturingService
database OrderRepository
database ManufacturingOrderRepository
ADMIN_KITCHEN ->> ManufacturingController : createManufacturingOrder()
ManufacturingController ->> ManufacturingService : generatePlan()
ManufacturingService ->> OrderRepository : findApprovedOrders()
OrderRepository -->> ManufacturingService : Approved Orders
ManufacturingService ->> ManufacturingOrderRepository : save(manufacturingOrder)
ManufacturingOrderRepository -->> ManufacturingService : Saved
ManufacturingService -->> ManufacturingController : Create Success
ManufacturingController -->> ADMIN_KITCHEN : Manufacturing Order Created
@enduml

```

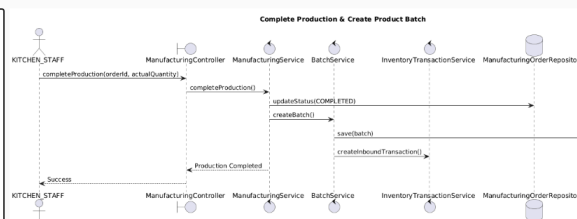


### 5. SEQUENCE 5 — Complete Production & Create Batch

```

@startuml
title Complete Production & Create Product Batch
actor KITCHEN_STAFF
boundary ManufacturingController
control ManufacturingService
control BatchService
control InventoryTransactionService
database ManufacturingOrderRepository
database BatchRepository
KITCHEN_STAFF ->> ManufacturingController : completeProduction(orderId, actualQuantity)
ManufacturingController ->> ManufacturingService : completeProduction()
ManufacturingService ->> ManufacturingOrderRepository : updateStatus(COMPLETED)
ManufacturingService ->> BatchService : createBatch()
BatchService ->> BatchRepository : save(batch)
BatchService -->> ManufacturingController : Production Completed
ManufacturingController ->> KITCHEN_STAFF : Success
note right of KITCHEN_STAFF
    Batch Status: AVAILABLE
end note
BatchService ->> InventoryTransactionService : createInboundTransaction()
InventoryTransactionService ->> ManufacturingOrderRepository : 
ManufacturingController -->> KITCHEN_STAFF : Success
@enduml

```

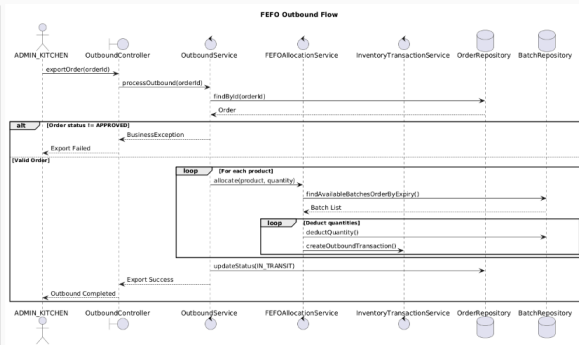


### 6. SEQUENCE 6 — FEFO Outbound Flow

```

@startuml
title FEFO Outbound Flow
actor ADMIN_KITCHEN
boundary OutboundController
control OutboundService
control FEFOAllocationService
control InventoryTransactionService
database OrderRepository
database BatchRepository
ADMIN_KITCHEN->>OutboundController: exportOrder(orderId)
OutboundController->>OutboundService: processOutbound(orderId)
OutboundService->>OrderRepository: findValidOrder(orderId)
OrderRepository-->>OutboundService: Order
alt Order status != APPROVED
OutboundService->>OutboundController: BusinessException
OutboundController-->>ADMIN_KITCHEN: Export Failed
else Valid Order
loop For each product
OutboundService->>FEFOAllocationService: allocate(product, quantity)
FEFOAllocationService->>BatchRepository: findAvailableBatchesOrderByExpiry()
BatchRepository-->>FEFOAllocationService: Batch List
loop Deduct quantities
FEFOAllocationService->>BatchRepository: deductQuantity()
FEFOAllocationService->>InventoryTransactionService: createOutboundTransaction()
end
end
OutboundService->>OrderRepository: updateStatus(DN_TRANSIT)
OutboundService-->>OutboundController: Export Success
OutboundController-->>ADMIN_KITCHEN: Outbound Completed
end
@enduml

```

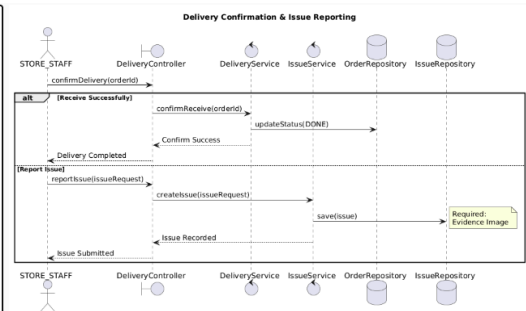


## 7. SEQUENCE 7 — Delivery Confirmation & Issue Reporting

```

@startuml
title Delivery Confirmation & Issue Reporting
actor STORE_STAFF
boundary DeliveryController
control DeliveryService
control IssueService
database OrderRepository
database IssueRepository
STORE_STAFF->>DeliveryController: confirmDelivery(orderId)
all Receive Successfully
DeliveryController->>DeliveryService: confirmReceive(orderId)
DeliveryService->>OrderRepository: updateStatus(DONE)
DeliveryService-->>DeliveryController: Confirm Success
DeliveryController-->>STORE_STAFF: Delivery Completed
else Report Issue
STORE_STAFF->>DeliveryController: reportIssue(issueRequest)
DeliveryController->>IssueService: createIssue(issueRequest)
IssueService->>IssueRepository: save(issue)
IssueService-->>DeliveryController: Issue Recorded
DeliveryController-->>STORE_STAFF: Issue Submitted
end
note right
    Required:  
Evidence Image
end note
@enduml

```



# Package Diagram

tiếp tục với Package Diagram

Tốt. Với project này, Package Diagram rất quan trọng vì system của bạn đã đủ lớn để cần modular architecture.

” Ask ChatGPT

Và khác với Sequence Diagram:

Package Diagram  
KHÔNG mô tả flow



nó mô tả:

system structure  
+  
module dependencies



## 1. Mục tiêu của Package Diagram

Ta muốn thể hiện:

### A. Các module nghiệp vụ

auth  
ordering  
inventory  
manufacturing  
delivery



### B. Dependency giữa modules



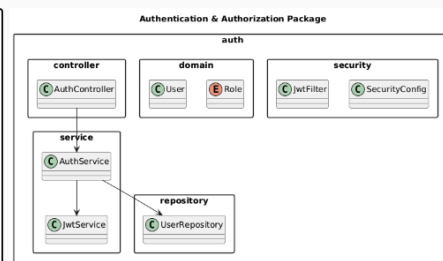
Ví dụ:

+ Ask anything



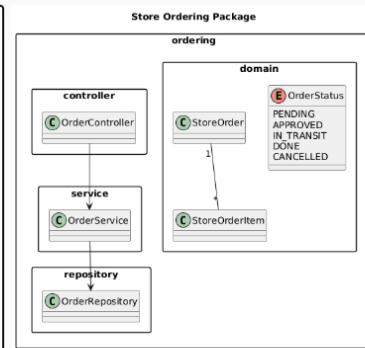
## 1. Package 1 - Authentication & Authorization

```
@startuml
title Authentication & Authorization Package
skinparam packageStyle rectangle
package "auth" {
    package "controller" {
        class AuthController
    }
    package "service" {
        class AuthService
        class JwtService
    }
    package "repository" {
        class UserRepository
    }
    package "domain" {
        class User
        class Role
    }
    package "security" {
        class JwtFilter
        class SecurityConfig
    }
}
AuthController --> AuthService
AuthService --> UserRepository
AuthService --> JwtService
@enduml
```



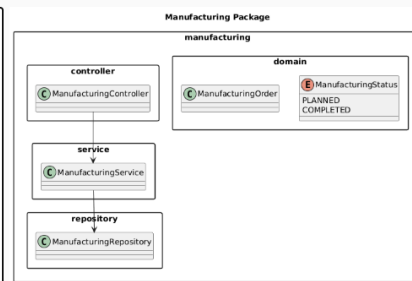
## 2. Package 2 - Store Ordering

```
@startuml
title Store Ordering Package
skinparam packageStyle rectangle
package "ordering" {
    package "controller" {
        class OrderController
    }
    package "service" {
        class OrderService
    }
    package "repository" {
        class OrderRepository
    }
    package "domain" {
        class StoreOrder
        class StoreOrderItem
        enum OrderStatus {
            PENDING
            APPROVED
            IN_TRANSIT
            DONE
            CANCELLED
        }
    }
}
OrderController --> OrderService
OrderService --> OrderRepository
StoreOrder "1" -- "*" StoreOrderItem
enduml
```



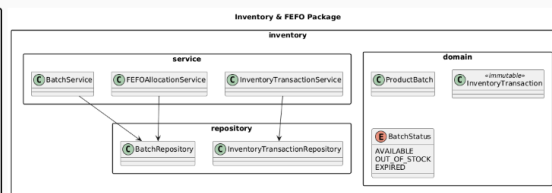
## 3. Package 3 - Manufacturing

```
@startuml
title Manufacturing Package
skinparam packageStyle rectangle
package "manufacturing" {
    package "controller" {
        class ManufacturingController
    }
    package "service" {
        class ManufacturingService
    }
    package "repository" {
        class ManufacturingRepository
    }
    package "domain" {
        class ManufacturingOrder
        enum ManufacturingStatus {
            PLANNED
            COMPLETED
        }
    }
}
ManufacturingController --> ManufacturingService
ManufacturingService --> ManufacturingRepository
enduml
```



## 4. Package 4 - Inventory & FEFO

```
@startuml
title Inventory & FEFO Package
skinparam packageStyle rectangle
package "inventory" {
    package "service" {
        class BatchService
        class FEFOAllocationService
        class InventoryTransactionService
    }
    package "repository" {
        class BatchRepository
        class InventoryTransactionRepository
    }
    package "domain" {
        class ProductBatch
        class InventoryTransaction <<immutable>>
        enum BatchStatus {
            AVAILABLE
            OUT_OF_STOCK
            EXPIRED
        }
    }
}
BatchService --> BatchRepository
FEFOAllocationService --> BatchRepository
InventoryTransactionService --> InventoryTransactionRepository
enduml
```

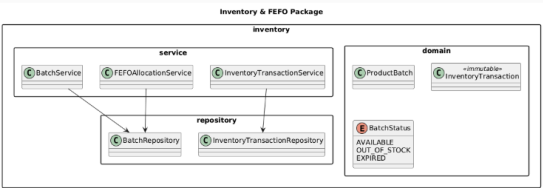


## 5. Package 5 - Delivery & Issue Handling

```

@startuml
title Inventory & FEFO Package
skinparam packageStyle rectangle
package "Inventory" {
    package "service" {
        class BatchService
        class FEFOAllocationService
        class InventoryTransactionService
    }
    package "repository" {
        class BatchRepository
        class InventoryTransactionRepository
    }
    package "domain" {
        class ProductBatch
        class InventoryTransaction <<immutable>>
        enum BatchStatus {
            AVAILABLE
            OUT_OF_STOCK
            EXPIRED
        }
    }
}
BatchService --> BatchRepository
FEFOAllocationService --> BatchRepository
InventoryTransactionService --> InventoryTransactionRepository
end

```

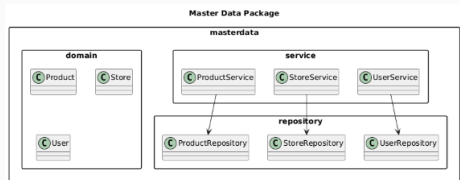


## 6. Package 6 - Master Data

```

@startuml
title Master Data Package
skinparam packageStyle rectangle
package "MasterData" {
    package "domain" {
        class Product
        class Store
        class User
    }
    package "repository" {
        class ProductRepository
        class StoreRepository
        class UserRepository
    }
    package "service" {
        class ProductService
        class StoreService
        class UserService
    }
}
ProductService --> ProductRepository
StoreService --> StoreRepository
UserService --> UserRepository
end

```



# Source code



# Tài Liệu: Các Luồng Nghiệp Vụ Hệ Thống Đổi Pin (Battery Swap System)

Tài liệu này tổng hợp lại các đoạn mã C# từ tài liệu gốc, được làm sạch, định dạng cấu trúc chuẩn mực, và sắp xếp gọn gàng cho các quy trình Đặt lịch, Hủy gói cước, Trả pin (Swap In) và Lấy pin (Swap Out).

## 1. Luồng Đặt lịch đổi pin (Create Booking)

### 1.1. BookingController.cs (API Endpoint)

Endpoint sử dụng method POST và bảo vệ bằng [Authorize(Roles = "Driver")] để đảm bảo chỉ có tài xế mới được phép tạo lịch đặt trước.

```
[Authorize(Roles = "Driver")]
[HttpPost("create-booking")]
public async Task<IActionResult> CreateBooking([FromBody] CreateBookingRequest request)
{
    // Gọi Service xử lý logic và nhận về ServiceResult
    var result = await _bookingService.CreateBookingAsync(request);

    // Trả về HTTP Status tương ứng và payload dữ liệu
    return StatusCode(result.Status, new
    {
        result.Status,
        result.Message,
        result.Data
    });
}
```

### 1.2. BookingService.cs (Business Logic)

Nơi xử lý logic chính cho việc tạo đặt lịch. Bao gồm kiểm tra tính hợp lệ, tính phí booking, khởi tạo lịch hẹn (Appointment), khóa Pin tại trạm (Lock slot) và lên lịch tự động hủy nếu quá hạn.

```
public async Task<ServiceResult> CreateBookingAsync(CreateBookingRequest request)
{
    // 1. Kiểm tra gói cước (Subscription) của người dùng có tồn tại không
    var subscription = await GetSubscriptionById(request.SubscriptionId);
```

```

    if (subscription == null)
        return new ServiceResult { Status = 404, Message = "Subscription not found" };

    // 2. Kiểm tra tài xế có đang có một booking nào đang "Processing" không
    (Tránh đặt trùng)
    var existingBooking = await bookingRepo.GetByIdAsync(b =>
        b.SubscriptionId == request.SubscriptionId && b.Status == "Processing");

    if (existingBooking != null)
        return new ServiceResult { Status = 409, Message = "You already have an active booking. Please complete or cancel it before creating a new one." };

    // 3. Lấy thông tin phí đặt lịch (Booking Fee) theo gói cước (Plan)
    var getFee = await unitOfWork.Fees.GetByIdAsync(f =>
        f.PlanId == subscription.PlanId && f.TypeOfFee == "Booking");

    // 4. Xử lý Giao dịch (Transaction) phí đặt lịch
    var gettrans = await unitOfWork.Trans.GetByIdAsync(t =>
        t.SubscriptionId == request.SubscriptionId &&
        t.TransactionType == "Monthly Fee" && t.Status == "Waiting");

    string newTransId = await GenerateTransactionId();

    if (gettrans == null) {
        // Nếu chưa có giao dịch chờ, tạo mới Giao dịch ghi nhận phí booking
        var newTransaction = new Transaction { /* Khởi tạo các trường cần thiết... */ };
        await _unitOfWork.Trans.CreateAsync(newTransaction);
    } else {
        // Nếu đã có giao dịch chờ, cộng dồn phí booking vào tổng phí
        gettrans.Fee += getFee.Amount;
        gettrans.TotalAmount += getFee.Amount;
        await _unitOfWork.Trans.UpdateAsync(gettrans);
    }

    await unitOfWork.SaveChangesAsync();

    // 5. Khởi tạo đối tượng Lịch hẹn (Appointment) mới
    string bookingId = await GenerateBookingId();
    var appointmentDB = new Appointment
    {

```

```

        AppointmentId = bookingId,
        UserDriverId = request.DriverId,
        BatterySwapStationId = request.StationId,
        SubscriptionId = request.SubscriptionId,
        Status = "Processing",
        DateBooking = request.DateBooking,
        TimeBooking = request.TimeBooking,
        CreateBookingAt = DateTime.UtcNow.ToLocalTime()
    };

    await _bookingRepo.CreateAsync(appointmentDB);
    await unitOfWork.SaveChangesAsync();

    // 6. TRỌNG TÂM: Gọi sang PillarSlotService để giữ chỗ (Lock) các cục pin khả
    dụng tại Trạm
    var lockedSlot = await _pillarSlotService.LockSlotsAsync(request.StationId,
    request.SubscriptionId, bookingId);

    // 7. Nếu trạm hết pin phù hợp để giữ chỗ -> Hủy Booking & Hoàn tiền
    (Rollback logic)
    if (lockedSlot == null || !lockedSlot.Any())
    {
        appointmentDB.Status = "Canceled";
        // Logic trừ lại tiền transaction đã cộng phía trên
        await _unitOfWork.SaveChangesAsync();
        return new ServiceResult { Status = 409, Message = "There is no pillar in
    this station with enough available batteries..." };
    }

    // 8. Tính toán thời gian quá hạn và thiết lập Background Task tự động hủy
    booking
    var calculatetime = CalculateCancelCountdownSeconds(appointmentDB);
    ScheduleAutoCancel(appointmentDB.AppointmentId, calculatetime);

    // 9. Trả về Response thành công cho Frontend
    return new ServiceResult
    {
        Status = 201,
        Message = $"Locked {lockedSlot.Count} batteries at pillar
    {lockedSlot.FirstOrDefault()?.PillarId}.",
        Data = new { Booking = appointmentDB, Time = calculatetime }
    };

```

```
}
```

### 1.3. Các hàm Helper hỗ trợ trong Service

Để đoạn code trên hoạt động hoàn hảo, hệ thống gọi các hàm sinh mã tự động và đặc biệt là cơ chế chạy ngầm (Background Task) để tự động hủy đơn sau khi quá giờ.

```
// Hàm tạo mã Booking tự động (VD: AP-1234567)
public async Task<string> GenerateBookingId()
{
    string bookingId;
    bool isDuplicated;
    do
    {
        bookingId = $"AP-{RandomNumberGenerator.GetInt32(0, 10_000_000):D7}";
        isDuplicated = await bookingRepo.AnyAsync(b => b.AppointmentId ==
bookingId);
    } while (isDuplicated);

    return bookingId;
}

// Hàm gọi tiến trình ngầm (Task.Run) để hệ thống tự động đếm ngược và hủy đặt
lịch
private void ScheduleAutoCancel(string bookingId, int delaySeconds)
{
    _ = Task.Run(() => AutoCancelAfterDelay(bookingId, delaySeconds));
}

// Tiến trình đếm ngược và kích hoạt hủy đơn
private async Task AutoCancelAfterDelay(string bookingId, int delaySeconds)
{
    // Chờ đếm ngược
    await Task.Delay(TimeSpan.FromSeconds(delaySeconds));

    // Mở một Scope mới để lấy Dependency Injection vì hàm này chạy tách biệt với
Request ban đầu
    using (var scope = _scopeFactory.CreateScope())
    {
        var scopedBookingService =
scope.ServiceProvider.GetRequiredService<IBookingService>();
    }
}
```

```

        var req = new CancelBookingRequest { BookingId = bookingId };

        // Tự động gọi API hủy đơn
        await scopedBookingService.CancelBookingAsync(req);
    }
}

```

---

## 2. Luồng Hủy Gói Cước (Cancel Plan)

Luồng này cho phép nhân viên (Staff) xác nhận hủy gói cước của khách hàng. Hệ thống sẽ tính toán lại phí dựa trên tiền cọc pin (Battery Deposit) và các giao dịch chưa thanh toán, sinh ra phí phạt (Penalty) hoặc hoàn tiền (Refund), và cập nhật trạng thái pin.

### 2.1. API Endpoint (BatterySwapController.cs)

```

[AllowAnonymous] // Lưu ý: Nên đi điều chỉnh lại quyền phù hợp, ví dụ
[Authorize(Roles = "Staff")]
[HttpPost("create-cancel-plan")]
public async Task<IActionResult> CreateCancelPlan(CheckCancelPlanRequest
requestDto)
{
    var result = await batSwapService.CancelPlanAsync(requestDto);
    return StatusCode(result.Status, new
    {
        message = result.Message,
        data = result.Data
    });
}

```

### 2.2. Business Logic (BatterySwapService.cs)

```

public async Task<ServiceResult> CancelPlanAsync(CheckCancelPlanRequest
requestDto)
{
    // 1. Lấy thông tin trạm, gói cước và phí cọc pin (Battery Deposit)
    var getPlanId = await GetPlanIdBySubId(requestDto.SubId);
    var getFee = await _feeRepo.GetAllQueryable()
        .FirstOrDefaultAsync(fee => fee.PlanId == getPlanId && fee.TypeOfFee ==
"Battery Deposit");
}

```

```

        // Lấy lịch hẹn hủy gói cước của khách hàng
        var booking = await
unitOfWork.Bookings.GetBookingCancelBySubId(requestDto.SubId);

        // 2. Lấy các giao dịch chưa thanh toán (Not Pay) của gói cước để cần trừ
        var gettransactionNotPay = await
unitOfWork.Trans.TransactionListNotpayBySubId(requestDto.SubId);

        // 3. Trừ tiền cọc pin vào tổng số tiền chưa thanh toán
        gettransactionNotPay.TotalAmount = gettransactionNotPay.TotalAmount -
getFee.Amount;
        gettransactionNotPay.Status = "Pending";
        gettransactionNotPay.Note = $"{requestDto.SubId}-CANCEL-PLAN";

        // 4. Tính toán phí vượt quãng đường (Mileage Fee) dựa trên lịch sử dùng pin
        var getSessionList = await GenerateBatterySession(requestDto.SubId);
        var calMilleageFee = await CalMilleageFee(getBatRequest, getSessionList);

        // 5. Xác định là Phạt (Penalty) hay Hoàn tiền (Refund)
        var total = gettransactionNotPay.TotalAmount - getFee.Amount;
        if (total > 0) {
            gettransactionNotPay.TransactionType = "Penalty Fee";
        } else {
            gettransactionNotPay.TransactionType = "Refund";
            gettransactionNotPay.PaymentMethod = "Cash";
        }

        await _transRepo.UpdateAsync(gettransactionNotPay);

        // 6. Hoàn tất Booking hủy gói cước
        if (booking != null) {
            booking.TransactionId = gettransactionNotPay.TransactionId;
            booking.Status = "Done";
            await appointmentRepo.UpdateAsync(booking);
        }
        await unitOfWork.SaveChangesAsync();

        // 7. Thu hồi toàn bộ pin mà khách hàng đang giữ (Đổi Status sang Returned)
        var batteriesInSub = await
unitOfWork.BatterySwap.GetBatteriesBySubscriptionId(requestDto.SubId);

```

```

        foreach (var bat in batteriesInSub) {
            bat.Status = "Returned";
            await unitOfWork.BatterySwap.UpdateAsync(bat);
        }
        await _unitOfWork.SaveChangesAsync();

        return new ServiceResult
        {
            Status = 200,
            Message = "Check confirm and refund for customer",
            Data = gettransactionNotPay
        };
    }
}

```

---

### 3. Luồng Trả pin vào trạm (Swap In / Check Battery Available)

Đây là bước đầu tiên khi tài xế đến trạm đổi pin. Hệ thống tiếp nhận pin cũ của tài xế xác minh, ghi lại lịch sử đổi trả, chuyển pin sang trạng thái sạc và gợi ý các cục pin mới để tài xế có thể lấy (Swap Out).

#### 3.1. API Endpoint (BatterySwapController.cs)

```

[Authorize(Roles = "Driver")]
[HttpPost("swap-in-battery")]
public async Task<IActionResult> SwapInBattery([FromBody] BatterySwapListRequest request)
{
    var result = await batSwapService.CheckBatteryAvailable(request);
    return StatusCode(result.Status, new
    {
        message = result.Message,
        data = result.Data
    });
}

```

#### 3.2. Business Logic (BatterySwapService.cs)

```

public async Task<ServiceResult> CheckBatteryAvailable(BatterySwapListRequest requestBatteryList)
{

```

```

    // 1. Kiểm tra tính hợp lệ của Pin (Xác minh pin đưa vào đúng là pin tài xế
    đang giữ)
    var getUnavailableBattery = await GetUnavailableBattery(
        requestBatteryList.AccessRequest.SubscriptionId,
        requestBatteryList.BatteryDtos);

    if (getUnavailableBattery != null && getUnavailableBattery.Any()) {
        return new ServiceResult { Status = 404, Message = "You're trying to
        return a different battery" };
    }

    var getSub = await
    subRepo.GetByIdAsync(requestBatteryList.AccessRequest.SubscriptionId);

    // Lấy lịch sử mượn pin ("Using") để update
    var swapOutHistories = await batSwapRepo.GetAllAsync(
        predicate: b => batteryIds.Contains(b.BatteryOutId) && b.Status ==
    "Using",
        asNoTracking: false);

    // 2. Tính toán và cộng dồn phí hao mòn quãng đường (Mileage Fee) vào Giao
    dịch chờ
    var getTransaction = await
    unitOfWork.Trans.GetAllQueryable().FirstOrDefaultAsync(...);
    var calMilleageFee = await CalMilleageFee(...);
    getTransaction.Fee += calMilleageFee;
    await unitOfWork.Trans.UpdateAsync(getTransaction);

    // 3. Xử lý trả pin vào các Khe (Pillar Slot)
    foreach (var item in requestBatteryList.BatteryDtos)
    {
        var getSlot = await unitOfWork.BatterySwap.GetPillarSlot(item.SlotId);
        var updateBat = await
        unitOfWork.Batteries.FindingBatteryById(item.BatteryId);

        // Đóng lịch sử cũ (Ghi chú: Giả định được xử lý tương tự)
        // swapOutHistory.Status = "Returned";

        // Ghi nhận lịch sử mới (Pin trả vào)
        var updateBatSwapIn = new BatterySwap
        {

```



```

        SwapHistoryId = await GenerateBatterySwapId(),
        BatteryInId = item.BatteryId,
        Status = "Returned",
        /*...*/
    };

    await batSwapRepo.CreateAsync(updateBatSwapIn);

    // Cập nhật khe cắm (Đã có pin, khóa lại để sạc)
    getSlot.BatteryId = item.BatteryId;
    getSlot.AppointmentId = null;
    getSlot.PillarStatus = "Unavailable";

    // Cập nhật pin (Đang sạc, Reset SOC)
    updateBat.BatteryStatus = "Charging";
    updateBat.Soc = Random.Shared.Next(1, 31); // Fake SOC sạc
    updateBat.BatterySwapStationId =
requestBatteryList.AccessRequest.StationId;
    }

    // 4. Tạo Battery Session mới và cộng dồn Current Mileage / Số lần Swap
    await unitOfWork.BatSession.BulkCreateAsync(getSessionList);
    getSub.CurrentMileage += getMilleageBase.Sum(x => x.MilleageBase);
    getSub.RemainingSwap += numberOfBattery;

    await subRepo.UpdateAsync(getSub);
    await unitOfWork.SaveChangesAsync();

    // 5. Tìm và trả về danh sách các khe pin Đầy (Available) để FE mở cho tài
xế lấy ra
    var getPillarSlotList = await
unitOfWork.Stations.GetBatteriesAvailableByPillarIdAsync(...);

    return new ServiceResult
    {
        Status = 200,
        Data = new BatteryRequest { BatteryDtos = dtoList /* Danh sách pin khả
dụng */ }
    };

```

```
}
```

---

## 4. Luồng Lấy pin từ trạm (Swap Out)

Tiếp nối luồng Swap In, sau khi tài xế đẩy pin cũ vào trạm, hệ thống sẽ đẩy thông tin các pin sạc đầy ra. Tài xế chọn và lấy pin mới khỏi trạm.

### 4.1. API Endpoint (BatterySwapController.cs)

```
[Authorize(Roles = "Driver")]
[HttpPost("swap-out-battery")]
public async Task<IActionResult> SwapOutBattery([FromBody]
BatterySwapOutListRequest request)
{
    var result = await batSwapService.SwapOutBattery(request);
    return StatusCode(result.Status, new
    {
        message = result.Message,
        data = result.Data
    });
}
```

### 4.2. Business Logic (BatterySwapService.cs)

```
public async Task<ServiceResult> SwapOutBattery(BatterySwapOutListRequest
requestDto)
{
    // 1. Kiểm tra nếu có đơn Booking giữ chỗ thì hoàn thành (Done)
    bool checkBooking = await
bookService.CheckBookingExist(requestDto.SubscriptionId);
    if (checkBooking)
    {
        var booking = await unitOfWork.Bookings.GetAllQueryable()
            .Where(x => x.Status == "Processing" && x.SubscriptionId ==
requestDto.SubscriptionId)
            .FirstOrDefaultAsync();

        if (booking != null) {
            booking.Status = "Done";
        }
    }
}
```

```

        await appointmentRepo.UpdateAsync(booking);
    }
}

var swappedBatteries = new List<string>();

// 2. Xử lý logic rút pin (Swap Out) khỏi từng khe
foreach (var batteryDto in requestDto.BatteryDtos)
{
    var pillarEntity = await slotRepo.GetByIdAsync(x => x.SlotId ==
batteryDto.SlotId);
    var batteryEntity = await batRepo.GetByIdAsync(b => b.BatteryId ==
batteryDto.BatteryId);

    // Chỉ cho lấy pin khi khe đang báo Unavailable hoặc Lock
    if (pillarEntity != null && batteryEntity != null &&
        (pillarEntity.PillarStatus == "Unavailable" ||
pillarEntity.PillarStatus == "Lock"))
    {
        // Cập nhật trạng thái pin (Rời trạm, chuyển sang Using)
        batteryEntity.BatterySwapStationId = null;
        batteryEntity.BatteryStatus = "Using";

        // Giải phóng khe cắm (Trống)
        pillarEntity.BatteryId = null;
        pillarEntity.PillarStatus = "Available";

        // Ghi nhận lịch sử Swap (Pin lấy ra)
        var swapOut = new BatterySwap
        {
            SwapHistoryId = await GenerateBatterySwapId(),
            SubscriptionId = subId, // assumed variable context
            BatterySwapStationId = stationId, // assumed variable context
            BatteryOutId = batteryDto.BatteryId,
            Status = "Using",
            SwapDate = DateOnly.FromDateTime(DateTime.Today)
        };
        swappedBatteries.Add(batteryDto.BatteryId);

        await batSwapRepo.CreateAsync(swapOut);
        await batRepo.UpdateAsync(batteryEntity);
    }
}

```

```
        await slotRepo.UpdateAsync(pillarEntity);
    }
}

// 3. Tăng số đếm tổng số lần đổi pin của gói cước
var sub = await _subRepo.GetByIdAsync(subId);
if (sub != null) {
    sub.RemainingSwap += swappedBatteries.Count;
    await _subRepo.UpdateAsync(sub);
}

await unitOfWork.SaveChangesAsync();

// 4. Trả về Response chứa hóa đơn sau khi Swap out
return new ServiceResult
{
    Status = 200,
    Message = "Battery swapped out successfully",
    Data = new BillAfterSwapOutResponse { /* ... */ }
};
}
```